ACCESO A DATOS (2º DAM) • UT4 - PERSISTENCIA

Sistema de Inteligencia Táctica HoloNet

Persistencia Polimórfica con Hibernate 6, PostgreSQL y Consultas HQL Avanzadas

---

ALUMNO / DESARROLLADOR

Antonio Cano Ruiz

ESTRATEGIA DE MAPEO

InheritanceType.SINGLE_TABLE

El Desfase Objeto-Relacional (Impedance Mismatch)

¿Cómo persistir una jerarquía POO en tablas relacionales?

- **El Problema:** Las bases de datos relacionales no soportan herencia nativa de clases (cazas y destructores derivan de `Nave`).
- **La Solución:** Patrón `SINGLE_TABLE` (Tabla Única). Todas las entidades hijas conviven en la misma tabla física `naves`.
- **Ventaja Competitiva:** Máxima velocidad de lectura. Cero operaciones `JOIN` entre tablas jerárquicas y cero `UNION`.
- **Coste de Diseño:** Los campos específicos de cada subclase deben admitir valores `NULL`.

> **Clave ORM:** Hibernate utiliza una **columna discriminadora** para instanciar la clase exacta en tiempo de ejecución.

PostgreSQL Console - DDL Desnormalizado de Base de Datos

```
antonio@starfleet-station:~/Examen$ psql -U antonio -d hangar_imperio_db -f schema_init.sql

-- Limpieza preventiva de entidades existentes
DROP TABLE IF EXISTS naves;
DROP TABLE IF EXISTS planetas;

-- Creación de la tabla independiente 'planetas'
CREATE TABLE planetas (
    planeta_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    clima VARCHAR(50),
    afiliacion VARCHAR(50) DEFAULT 'Neutral'
);

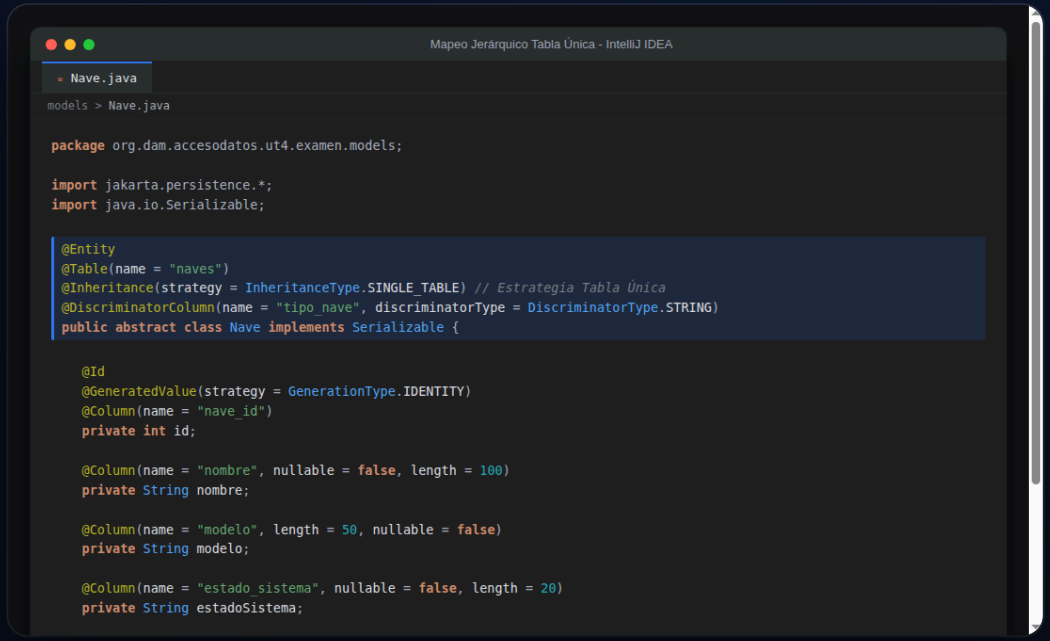
-- Creación de la tabla polimórfica 'naves' (Estrategia SINGLE_TABLE)
CREATE TABLE naves (
    nave_id SERIAL PRIMARY KEY,
    tipo_nave VARCHAR(30) NOT NULL,          -- Columna Discriminadora ('CAZA' / 'DESTRUCTOR')
    nombre VARCHAR(100) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    estado_sistema VARCHAR(20) NOT NULL,     -- 'Critico', 'Estable', 'Mantenimiento'
    planeta_origen_id INT,
    potencia_fuego INT,                      -- Específico de CazaLigero (adpite NULL)
    capacidad_escudos INT,                   -- Específico de DestructorEstelar (admite NULL)
    CONSTRAINT fk_planeta FOREIGN KEY (planeta_origen_id)
        REFERENCES planetas(planeta_id) ON DELETE SET NULL
);
✔ CREATE TABLE planetas OK
```

Diseño Relacional Desnormalizado

Estructura física de las tablas `planetas` y `naves`

- `tipo_nave VARCHAR(30)` Columna discriminadora obligatoria (valores: `CAZA` o `DESTRUCTOR`).
- `potencia_fuego INT` Específico de cazas (`NULL` para destructores).
- `capacidad_escudos INT` Específico de destructores (`NULL` para cazas).
- `CONSTRAINT fk_planeta` Clave foránea hacia `planetas(planeta_id)` con `ON DELETE SET NULL`.

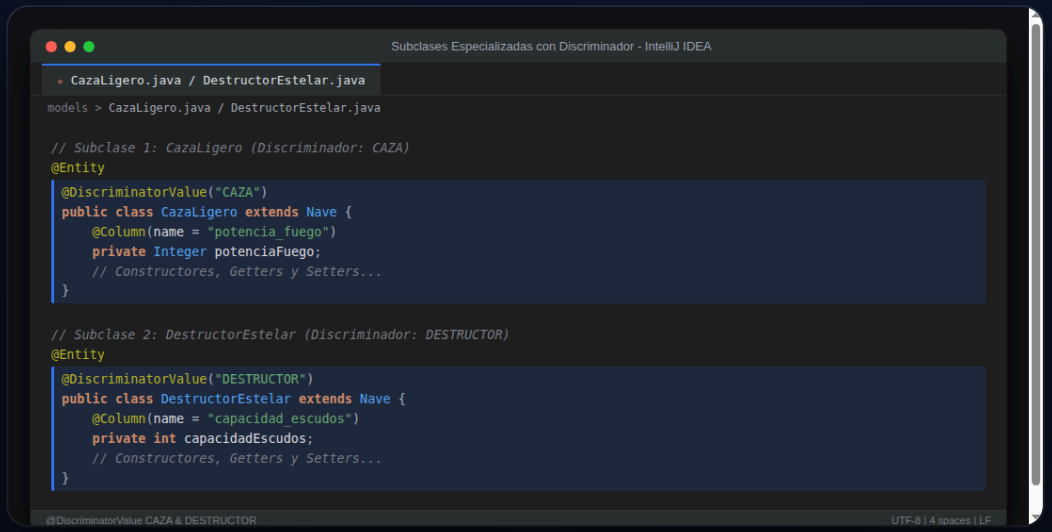
> Se insertaron 3 planetas y 5 naves cubriendo estados `Estable`, `Critico` y `Mantenimiento` para las pruebas de inteligencia.



Jerarquía de Clases con JPA

Anotaciones de persistencia y relaciones controladas

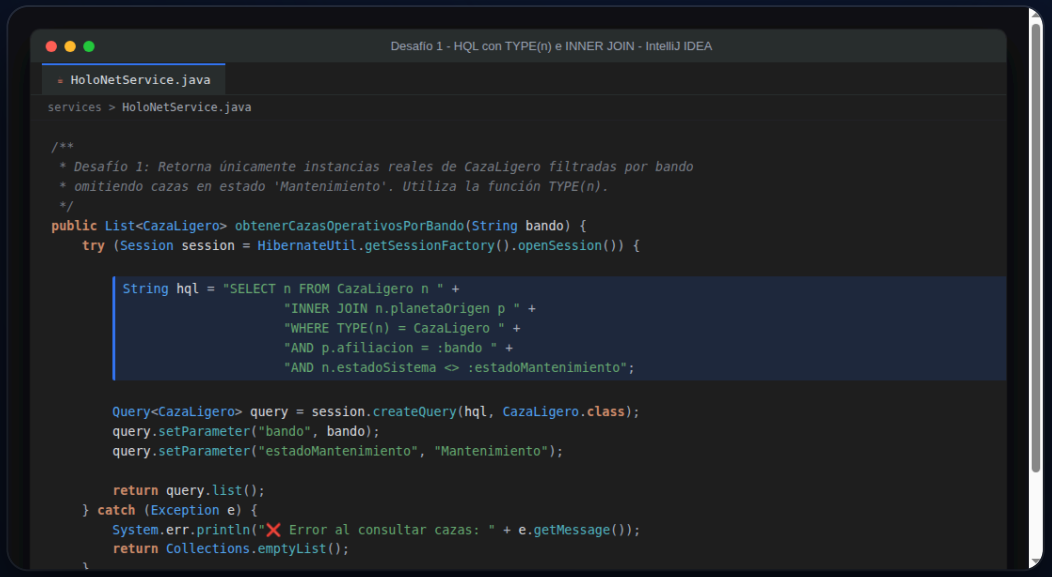
- `@Inheritance(strategy = SINGLE_TABLE)`: Informa a Hibernate que unifique la tabla física.
- `@DiscriminatorColumn(name = "tipo_nave")`: Define el campo de discriminación.
- `@ManyToOne(fetch = FetchType.LAZY)`: Carga diferida obligatoria hacia `Planeta` para optimizar la memoria.
- `@DiscriminatorValue("CAZA")` y `("DESTRUCTOR")` en las clases especializadas `CazaLigero` y `DestructorEstelar`.



Sensor de Escuadrones Polimórfico

Filtrado dinámico por tipo y relación de afiliación

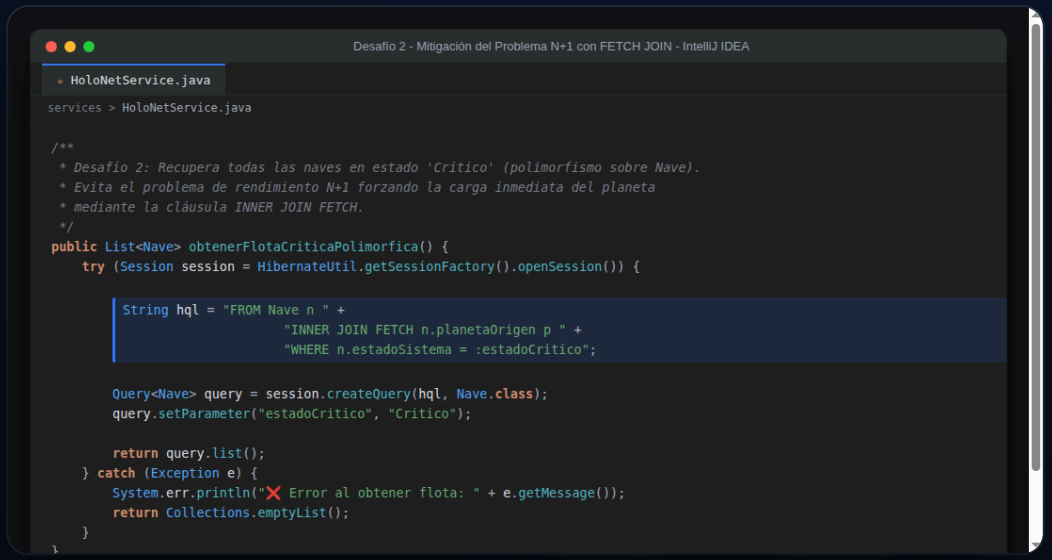
- **Objetivo:** Recuperar solo instancias de `CazaLigero` operativas asignadas a un bando determinado.
- **Uso de `TYPE(n)`:** Permite comprobar la clase polimórfica directamente en la consulta HQL (`TYPE(n) = CazaLigero`).
- **Filtro Táctico:** `n.estadoSistema <> :estadoMantenimiento` excluye naves no operativas (como el Halcón Milenario).
- **Navegación:** `INNER JOIN n.planetaOrigen p` filtra por `p.afiliacion = :bando`.



Mitigación del Problema N+1

Carga impaciente bajo demanda con `INNER JOIN FETCH`

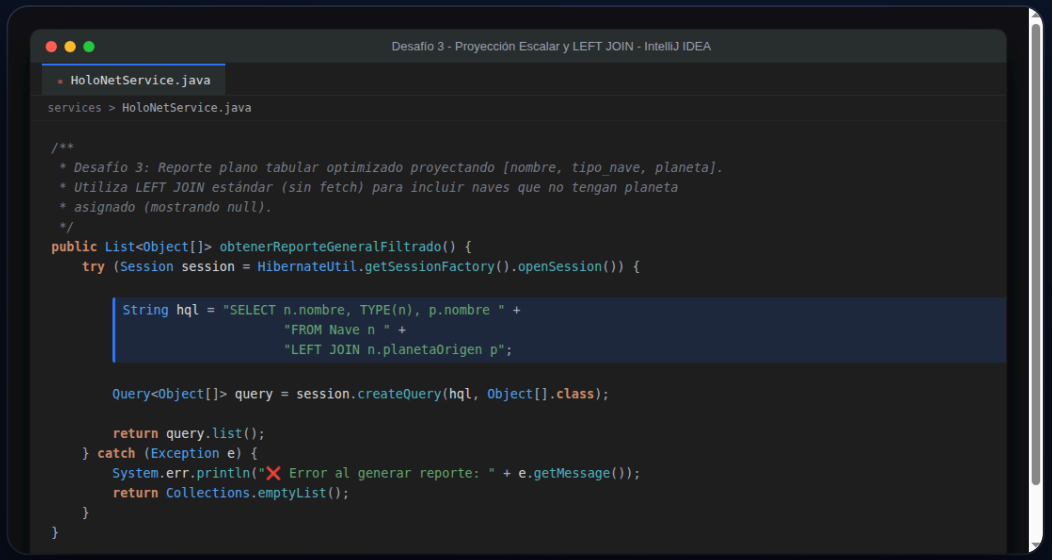
- **El Problema N+1:** Al tener `FetchType.LAZY`, acceder al planeta de cada nave crítica dispararía 1 query inicial + N consultas secundarias.
- **La Solución HQL:** `INNER JOIN FETCH n.planetaOrigen p`.
- **Efecto en SQL:** Hibernate genera una **única consulta SQL** con `JOIN`, poblando ambos objetos en memoria simultáneamente (Single Round-Trip).
- **Polimorfismo Real:** Recupera naves tanto destructores como cazas en estado `'Critico'`.



Proyecciones Planas de Inteligencia

Optimización de memoria y uso de `LEFT JOIN`

- **Retorno `List<Object[]>`:** Proyecta sólo 3 columnas: `n.nombre`, `TYPE(n)` y `p.nombre`.
- **Cero Sobrecarga:** No sobrecarga el contexto de persistencia con el seguimiento de estado (Dirty Checking).
- **Preservación con `LEFT JOIN`:** Si una nave no tuviera asignada base planetaria (`NULL`), no se descarta del reporte táctico.
- **Identidad POO:** `TYPE(n)` proyecta la clase Java concreta instanciada.

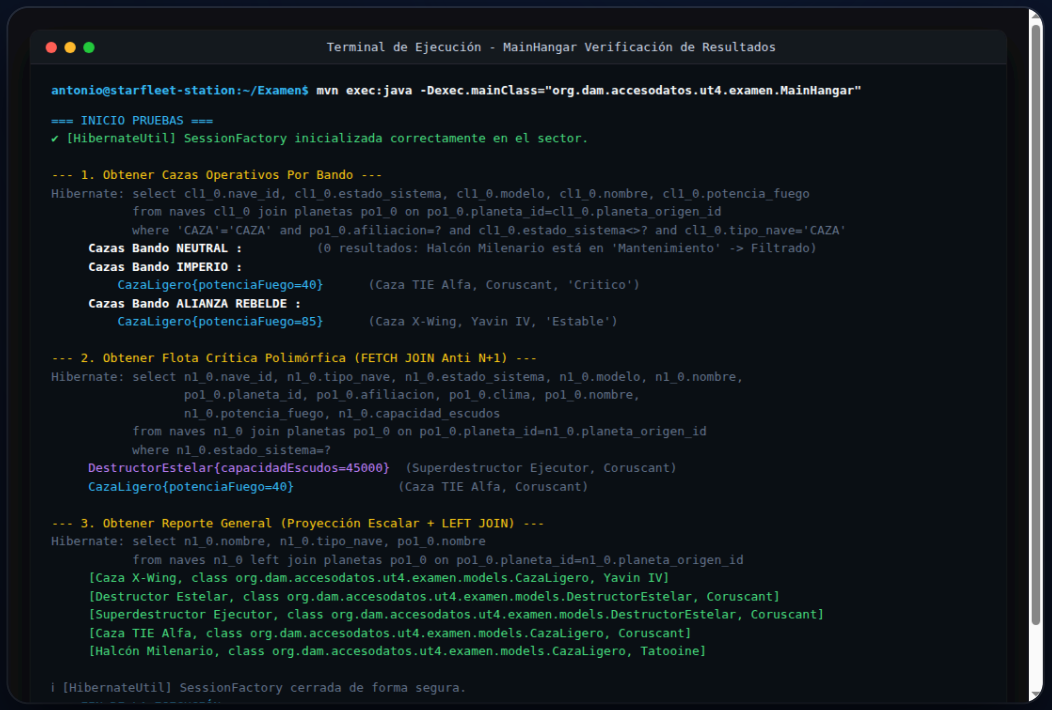


Validación y Salida por Consola

Auditoría del SQL generado y resultados en tiempo de ejecución

- **Prueba 1:** Neutral devuelve 0 cazas (Halcón en mantenimiento correctamente descartado). Imperio y Alianza devuelven sus respectivos cazas.
- **Prueba 2:** Recupera Superdestructor Ejecutor y Caza TIE Alfa con sus planetas precargados en 1 sola consulta SQL.
- **Prueba 3:** Reporte general plano con las 5 naves y su clase correspondiente.

> Gestión de conexiones segura garantizada con el patrón **try-with-resources**.



CONCLUSIONES DE LA PRÁCTICA

Persistencia Eficiente con Hibernate 6

- ✔ **SINGLE_TABLE:** La estrategia más veloz para jerarquías polimórficas de alta lectura.
- ✔ **FETCH JOIN:** La solución estándar para eliminar el problema N+1 sin perder los beneficios de LAZY.
- ✔ **HQL Avanzado:** Manejo de TYPE( ) y proyecciones escalares tabulares para optimizar memoria.

¡Muchas Gracias! ¿Preguntas?

Antonio Cano Ruiz • 2º DAM • Acceso a Datos